Table of contents

Workflow

App UI testing

Getting started

Installing and testing the Robo...

Show all...

Automated app testing overview

Roku app developers can use Roku's test automation software to write and execute test cases, including app purchasing, performance, deep linking, and other certification-related testing. Roku provides custom Selenium (https://selenium.dev)-based WebDriver APIs (/docs/developer-program/dev-tools/automated-channel-testing/web-driver.md) for sending commands to launch apps, send keypresses, and check whether SceneGraph components are present on the screen. Apps can use the WebDriver APIs to control a Roku device, while using a test framework or programming language to create, run, log, and record test cases. To make automated testing even easier, Roku provides Robot (/docs/developer-program/dev-tools/automated-channel-testing/robot-framework-library.md) and JavaScript (/docs/developer-program/dev-tools/automated-channel-testing/javascript-library.md) libraries and a Postman collection.

Executing test automation allows apps to run state-driven UI testing for a number of scenarios. For example, apps can create a test case that installs an app and launches it with a specific contentID and mediaType to verify that deep linking works. Authenticated apps can execute more complex test cases such as launching an app, trying to play content before authenticating the user, entering valid/invalid credentials, and then trying to play content again.

All test cases can be run simultaneously on multiple Roku devices. This is useful for testing app performance across different models with varying RAM and CPU. It is especially important for certification testing, which requires apps to meet performance criteria (/docs/developer-program/certification/certification.md#3-performance) that varies for different device types.

Implementing automated testing speeds up app development by reducing the number of manual UI tests that need to be run for simple to complex test cases.

Roku's test automation tools require Roku OS 9.1 or higher.

To test production apps with the Roku test automation tools, you must package the app (/docs/developer-program/publishing/packaging-channels.md#rekeying) on your Roku device using the same Roku developer account linked to the production version of the app.

Workflow

Test cases can be written with the Roku Robot Framework Library (/docs/developer-program/dev-tools/automated-channel-testing/robot-framework-library.md), Roku JavaScript library (/docs/developer-program/dev-tools/automated-channel-testing/javascript-library.md), another test framework, or a programming language such as Python, JavaScript, or Go. The test scripts send command requests to the Roku WebDriver (/docs/developer-program/dev-tools/automated-channel-testing/web-driver.md) via the JSON Wire Protocol (https://github.com/SeleniumHQ/selenium/wiki/JsonWireProtocol).

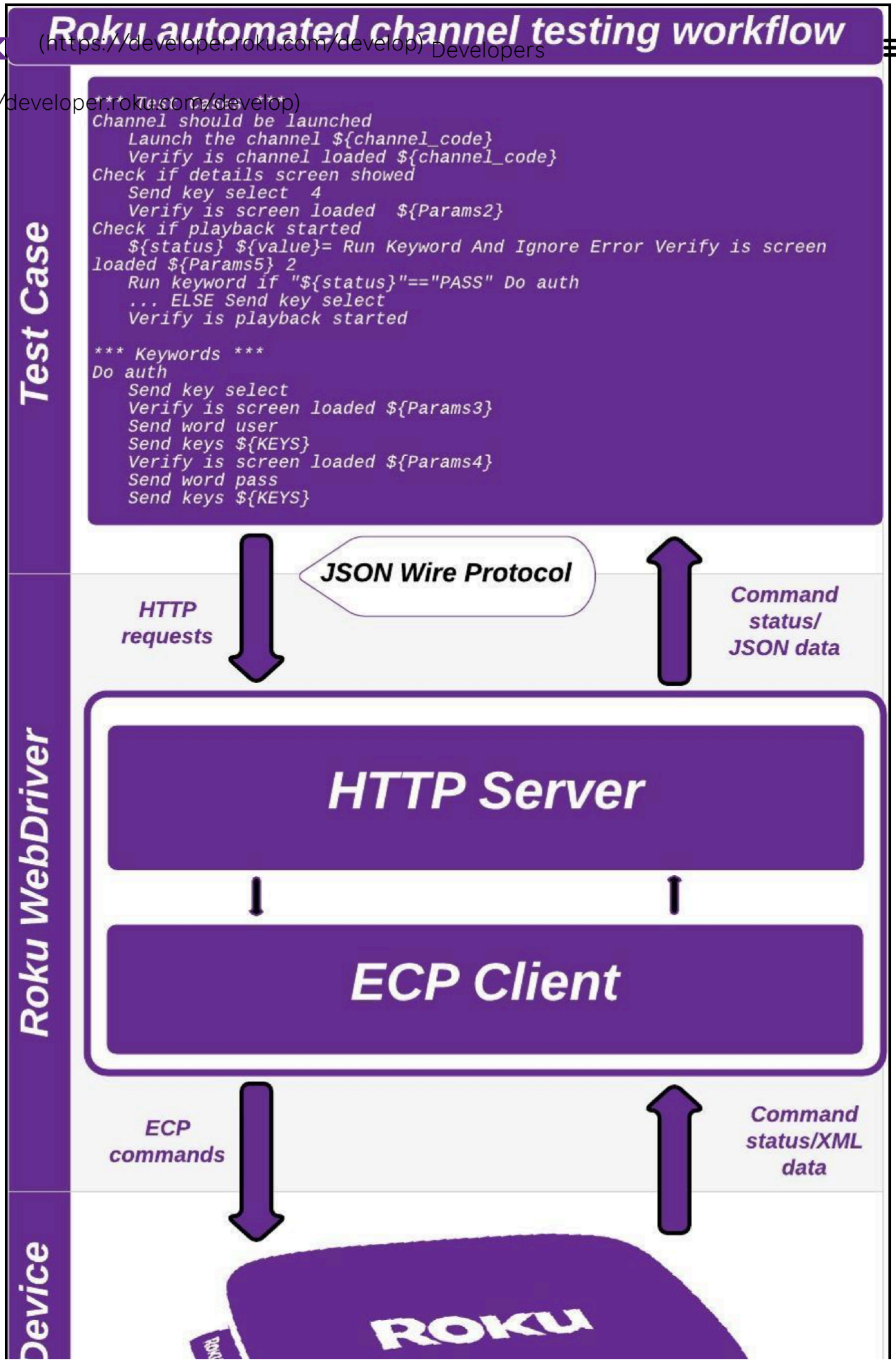
The Roku WebDriver includes an HTTP server that receives the command requests and an External Control Protocol (ECP) (/docs/developer-program/dev-tools/external-control-api.md) client that translates them into ECP requests, which it then routes to the target device.

The device returns the result of the command and XML data back to the Roku WebDriver, which then passes this information back to the test script as a JSON object (via the WebDriver client application).

The following diagram illustrates the automated app testing workflow:

Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)



App UI testing

The Roku WebDriver includes a set of APIs (/docs/developer-program/dev-tools/automated-channel-testing/web-driver.md#roku-webdriver-apis) that enable developers to run state-driven UI tests. For example, the RokuWebDriver has an element API (/docs/developer-program/dev-tools/automated-channel-testing/web-driver.md#POST-v1/session/:sessionId/element) to check if a specific SceneGraph component is present on the screen in order to determine whether a specific screen has been loaded. In addition, the Roku Robot Framework and JavaScript libraries have keywords (/docs/developer-program/dev-tools/automated-channel-testing/robot-framework-library.md#keywords) that are mapped to the Roku WebDriver APIs so that developers can execute app UI-based test cases with them.

Getting started

Test the Roku WebDriver (/docs/developer-program/dev-tools/automated-channel-testing/web-driver.md) following these steps:

1. Clone the Roku automated app testing repository (https://github.com/rokudev/automated-channel-testing) or download it as a zip file.

2. Run Roku's Python-based sample WebDriver client application following these steps:

   a. Download (https://www.python.org/downloads) and install Python 3.7 (or higher). Set the version you install as the default version of Python on your computer.

   b. Download and install the Python package installer (pip (https://pypi.org/project/pip)).

   c. Install the requests (https://pypi.org/project/requests) HTTP library for Python, which enables the sample client application to send HTTP 1.1 requests:

   ```
   python -m pip install requests
   ```

   d. Sideload (/docs/developer-program/getting-started/developer-setup.md#sideloading-channels) the sample app (channel.zip) included in the /automated-channel-testing-master/sample directory.

   e. Run the sample Web driver client application. Include the IP address of your Roku device as an argument. If the test is successful, "Test Passed" is output in the console.

```
python <path>/automated-channel-testing-master/sample/script/main.py <device-ip-address>
```


Installing and testing the Robot Framework Library

To install the Roku Robot Framework Library (/docs/developer-program/dev-tools/automated-channel-testing/robot-framework-library.md) and test it on one or more devices, follow these steps:

1. Optionally, install the Python version of the Roku Robot Framework Library via a local Python package. This enables you to directly import the Roku Robot Framework library in your Robot test case files.

   ```
   python pip install <path>/automated-channel-testing-master/RobotLibrary
   ```

2. Install the dependencies listed in the /automated-channel-testing-master/RobotLibrary/requirements.txt file:

   ```
   python -m pip install -r /automated-channel-testing-master/RobotLibrary/requirements.txt
   ```

3. Update line 41 of the /automated-channel-testing-master/RobotLibrary/Tests/Basic_tests.robot file with the password of your Roku device.

4. Run the sample basic Robot test case on a single device. When running the Robot tests and samples, you must run them from the RobotLibrary folder. You must also provide the Roku device IP address and WebDriver server path as variables in the console as demonstrated in the following example:

   ```
   cd RobotLibrary
   python -m robot.run --outputdir Results --variable ip_address:192.168.1.94 --variable server_path:
   ```

   Alternatively, you can hard code the Roku device IP address and WebDriver server path variables in the /automated-channel-testing-develop/RobotLibrary/Library/variables.py file, and then use the following command:

   ```
   python3 -m robot.run --outputdir Results Tests/Basic_tests.robot
   ```

5. View the generated test case report, which is stored in the specified output directory (/automated-channel-testing-master/RobotLibrary/Results by default).

Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

REPORT
Generated
20200319 09:10:31 UTC-07:00
29 minutes 57 seconds ago

Basic tests Log

Test Statistics

Total Statistics	Total	Pass	Fail	Elapsed	Pass / Fail
Critical Tests	3	3	0	00:00:14	
All Tests	3	3	0	00:00:14	

Statistics by Tag	Total	Pass	Fail	Elapsed	Pass / Fail
No Tags					

Statistics by Suite	Total	Pass	Fail	Elapsed	Pass / Fail
Basic tests	3	3	0	00:00:15	

Test Execution Log

SUITE **Basic tests** 00:00:14.575
Full Name: Basic tests
Documentation: Basic smoke tests
Source: /Users/jduval/Downloads/roku-automated-channel-testing-develop/RobotLibrary/Tests/Basic_tests.robot
Start / End / Elapsed: 20200319 09:10:16.773 / 20200319 09:10:31.348 / 00:00:14.575
Status: 3 critical test, 3 passed, 0 failed
3 test total, 3 passed, 0 failed

TEST **Channel should be launched** 00:00:00.011
Full Name: Basic tests.Channel should be launched
Start / End / Elapsed: 20200319 09:10:16.960 / 20200319 09:10:16.971 / 00:00:00.011
Status: PASS (critical)
KEYWORD RobotLibrary.**Verify is channel loaded** ${channel_code} 00:00:00.010

TEST **Check if details screen showed** 00:00:06.176
Full Name: Basic tests.Check if details screen showed
Start / End / Elapsed: 20200319 09:10:16.971 / 20200319 09:10:23.147 / 00:00:06.176
Status: PASS (critical)
KEYWORD RobotLibrary.**Send key** select, 4 00:00:04.062
KEYWORD RobotLibrary.**Verify is screen loaded** ${Params2} 00:00:02.113

TEST **Check if playback started** 00:00:08.200
Full Name: Basic tests.Check if playback started
Start / End / Elapsed: 20200319 09:10:23.147 / 20200319 09:10:31.347 / 00:00:08.200
Status: PASS (critical)
KEYWORD ${status}, ${value} = BuiltIn.**Run Keyword And Ignore Error** Verify is screen loaded, ${Params5}, 2 00:00:02.054
KEYWORD BuiltIn.**Run Keyword If** "${status}"=="PASS", Do auth, ELSE, Send key, select 00:00:02.042
KEYWORD RobotLibrary.**Verify is playback started** 20, 2 00:00:04.103

6. Run the sample basic Robot test case on multiple devices following these steps:

   a. Update the JSON configuration file (config.json) in the automated-channel-testing-master/RobotLibrary/multipleDevices directory, which contains the Roku devices to be used for testing, the Web driver server path, test case, and the output directory.

   Each Roku device is an object that has an arbitrary name and a key-value pair with the device's IP address. Key-value pairs may also be provided for the timeout and keypress delay to be used for the test on that device (these override the global test values specified in the Robot test case).

   The syntax of the config.json file is as follows:

```
{
    devices : {
        "Device 1 name": {
            "ip_address": <string>,
            "timeout":  <number>,
            "pressDelay": <number>
        },
        "Device 2 name": {
            "ip_address": <string>,
            "timeout":  <number>
        }
    },
    "server_path": <string>,
    "test": <string>,
    "outputdir": <string>
}
```

The following example demonstrates how to write the config.json file:

```
{
    "devices": {
        "Amarillo": {
            "ip_address": "192.168.1.64",
            "timeout":  20000,
            "pressDelay": 2000
        },
        "Littlefield": {
            "ip_address": 192.168.1.16,
            "timeout":  25000,
            "pressDelay": 1000
        }
    },
    "server_path": "/automated-channel-testing-master/bin/RokuWebDriver_<os|linux|windows.exe>
    "test": "Tests/Basic_tests_multi_device.robot",
    "outputdir": "Results"
}
```

b. Sideload (/docs/developer-program/getting-started/developer-setup.md#sideloading-channels) the sample app (channel.zip) in the /automated-channel-testing-master/sample folder.

c. Update the /automated-channel-testing-master/RobotLibrary/Library/variables.py file with the IP address of the Roku test device and WebDriver path.

d. Run the following console command:

```
cd RobotLibrary
python multipleDevices/multi.py multipleDevices/config.json
```

e. View the generated test case report and log for each device, which are stored in the specified output directory (/automated-channel-testing-master/RobotLibrary/Results by default).

Installing and testing the Roku JavaScript Library

To install the Roku JavaScript Library (/docs/developer-program/dev-tools/automated-channel-testing/javascript-library.md) and test it on one or more devices, follow these steps:

1. Download and install the node.js (https://nodejs.org/en/) JavaScript runtime environment.

2. Download and install the Yarn (https://classic.yarnpkg.com/en/docs/install) JavaScript package manager.

3. Install the dependencies listed in the /automated-channel-testing-master/jsLibrary/package.json file:

   ```
   yarn install
   ```

4. To use the Mocha (https://mochajs.org/) JavaScript test framework and run tests on multiple devices, globally install Mocha and Mochawesome (https://www.npmjs.com/package/mochawesome):

   ```
   yarn global add mocha
   yarn global add mochawesome
   ```

5. Update the /automated-channel-testing-master/jsLibrary/tests/test_basic.js file with the following:

   a. In line 20, update the WebDriver server path.

   b. In line 27, update the IP address to your Roku device.

   c. In line 28, update the password.

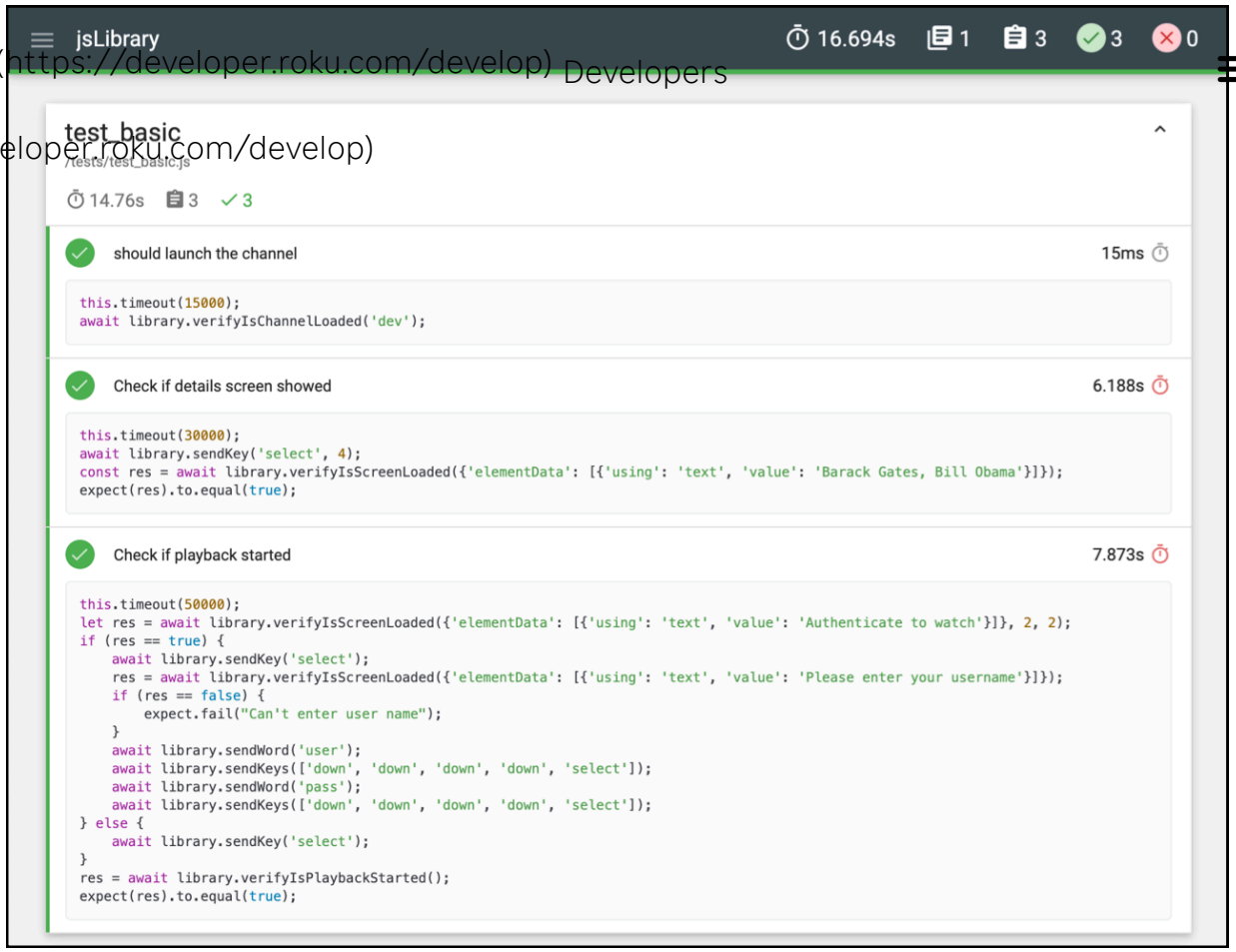
6. Run the sample basic JavaScript test case on a single device. When running the JavaScript tests and samples, you must run them from the jsLibrary folder

   ```
   yarn tests/test_basic.js
   ```

   To run the test using Mocha and report the test results with Mochawesome, enter the following command:

   ```
   mocha tests/test_basic.js --reporter mochawesome
   ```

7. View the generated test case report, which is stored in the mochawesome-report directory.



8. Run the sample basic JavaScript test case on multiple devices following these steps:

   a. Update the JSON configuration file (config.json) in the /automated-channel-testing-master/jsLibrary/multipleDevices directory, which contains the Roku devices to be used for testing, the Web driver server path, test case, and the output directory.

   Each Roku device is an object that has an arbitrary name and a key-value pair with the device's IP address. Key-value pairs may also be provided for the timeout and keypress delay to be used for the test on that device (these override the global test values specified in the Robot test case).

   The syntax of the config.json file is as follows:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
{
    "devices": {
        "Device 1 name": {
            "ip_address": <string>,
            "timeout":  <number>,
            "pressDelay": <number>
        },
        "Device 2 name": {
            "ip_address": <string>,
            "timeout":  <number>
        }
    },
    "server_path": <string>,
    "test": <string>,
    "outputdir": <string>
}
```

The following example demonstrates how to write the config.json file:

```
{
    "devices": {
        "Amarillo": {
            "ip_address": "192.168.1.64",
            "timeout":  20000,
            "pressDelay": 2000
        },
        "Littlefield": {
            "ip_address": 192.168.1.16,
            "timeout":  25000,
            "pressDelay": 1000
        }
    },
    "server_path": "/automated-channel-testing-master/bin/RokuWebDriver_<os|linux|windows.exe>
    "test": "multipleDevices/multiple_devices_test_basics.js",
    "outputdir": "Results"
}
```

b. Sideload (/docs/developer-program/getting-started/developer-setup.md#sideloading-channels) the sample app (channel.zip) in the /automated-channel-testing-master/sample folder.

c. Run the following console command:

```
node multipleDevices/multi.js  config.json
```

d. View the generated test case report and log for each device, which are stored in the specified output directory (/automated-channel-testing-master/jsLibrary/Results by default).

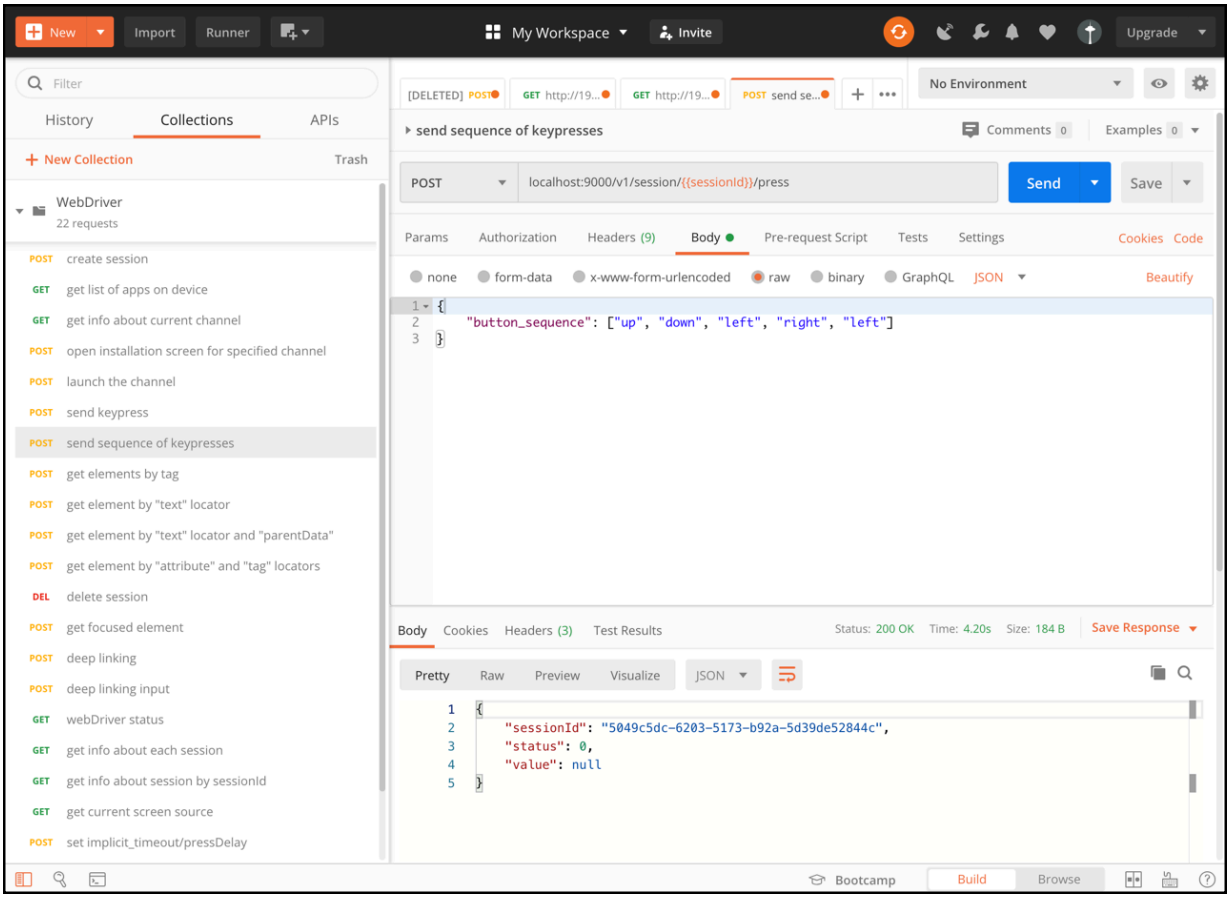
Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

Postman collection

To import the Postman JSON collection and use it to test the Roku WebDriver API calls, follow these steps:

1. Download (https://www.postman.com/downloads/) Postman.
2. Verify that the Roku WebDriver server is running (to start the WebDriver, run the main executable in the /automated-channel-testing-master/src folder).
3. Sideload (/docs/developer-program/getting-started/developer-setup.md#sideloading-channels) the sample app (channel.zip) in the /automated-channel-testing-master/sample folder.
4. In Postman, import the /automated-channel-testing-master/sample/Postman/WebDriver_endpoints Postman collection.
5. Create a new session. To do this, click the POST create session request, update the IP address to your Roku device, and then click Send.
6. Execute the requests in the Postman collection to test the Roku WebDriver.



7. When you have finished testing, send the DEL delete session request to remove the session.

Demo Video

The following video demonstrates the Roku automated app testing software. It provides a brief overview of the technology stack, and it shows how both the Roku WebDriver and Robot Framework Library can be used for state-driven app UI automation testing.

0:00 / 11:53

STB-tester

An alternative for using Roku's test automation software is the stb-tester, which is a small hardware device that you connect to your Roku device and use for executing automated test scripts. The stb-tester includes support for sideloading, launching, and navigating Roku apps and BrightScript debugging. Visit stb-tester.com (https://stb-tester.com/) to get more information on this device.

> The stb-tester is not a Roku product or in any way affiliated with Roku; however, some Roku developers do use it for automating their test processes.

Change log

v.2.3.0 (September 2021)

- Added option to run the Roku WebDriver on a specific port.
- Fixed password validation for "/load" endpoint.
- Updated Roku Recommends sample app.
- Adjusted sample tests that leverage Roku Recommends and SearchView apps.

v.2.2.0 (April 2021)

- Added pre-built WebDrivers for iOS, Linux, and Windows.
- Added option for installing Python version of Robot library as a local Python package.
- Fixed submitting '@' symbol over "Send word" keyword.
- Fixed "Get player info" keyword failing when called before playback starts.
- Adjusted sample tests for Robot and JS libraries (added Sideload command to automate sideloading of sample apps used for Robot and JavaScript sample tests).

v.2.1.0 (July 2020)

- Added "Get child nodes" method for Robot and JavaScript libraries.
- Updated WebDriver /element/active endpoint to consistently return correct element.
- Updated WebDriver /elements endpoint to returns correct elements when multiple locators are specified.
- Added and updated sample tests for Robot and JavaScript libraries.

v.2.0.0 (March 2020)

- Roku JavaScript test library.
- Postman collection.
- Multiple device support for Roku Robot and JavaScript libraries.
- App sideloading.
- Input deep linking.
- Timers.

v.1.0.0 (December 2019)

- Initial release of Roku automated app testing software.
- Initial Roku WebDriver version.

- Keypress simulation.
- UI element retrieval.
- Current app metadata.

(https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

Previous Next

Terms and Conditions

B2B Privacy Notice (https://docs.roku.com/published/b2bprivacypolicy/en/us)

Developer Site (https://developer.roku.com)

About ads & cookies (https://docs.roku.com/published/cookiepolicy)

My Privacy Choices
(https://privacy.roku.com/ccpa)
Developer Terms & Agreements (https://developer.roku.com/docs/features/legal/developer-terms.md)

CA B2B Privacy Notice

© 2008-2026 Roku, Inc. (https://www.roku.com)